

大模型训练与对齐：SFT、RLHF(PPO)、DPO、GRPO 与 Agentic RL

AI播客口播稿 + MiniMind 开源项目速览与上手指引

生成日期：2026-02-10 (America/Los_Angeles)

如何使用这份稿件

- 你可以直接将正文作为播客脚本朗读；每节标题就是换段提示。
- 如果你要做“记忆强化”，建议每节末尾暂停，用“自测问题”复述一遍。
- MiniMind 部分适合边听边动手：先跑评测，再逐步读训练脚本。

Part A AI播客口播稿

提示：建议1.05-1.15倍速收听；遇到公式与概念处可降速。

0. 开场：一张脑图

今天用一条训练链路把大模型对齐讲清楚：SFT → 对齐优化（RLHF/PPO 或 DPO/GRPO）→ Agentic RL。

一句话记忆：SFT教会模型“怎么说”；对齐阶段教会模型“说得让人满意”；Agentic RL教会模型“为了目标去做事”。

每个方法都按同一套结构学习：定义 → 优化目标直觉 → 工程落地 → 常见问题 → 解决策略 → 延展思考。

1. SFT：把“会说话”训练出来

定义：监督微调（Supervised Fine-Tuning），用(输入→理想输出)让模型最大化正确答案概率，本质是模仿学习。

工程四件套：数据（干净一致）/ 模板（system-user-assistant）/ loss mask（只算assistant）/ 评估（别只看loss）。

典型问题：啰嗦谄媚、过拟合格式、灾难性遗忘；常用解法：清洗与风格统一、指令多样化、混入通用数据与小学习率。

2. 对齐优化：从“会说”到“让人满意”

核心变化：不再只给“标准答案”，而是提供“偏好比较”（哪个更好）。

两大范式：RLHF(PPO)（重但灵活）与 DPO/GRPO（更轻更稳）。

2.1 RLHF（PPO）：经典三段式闭环

三步：SFT打底 → 训练奖励模型RM（偏好打分器）→ 用PPO在KL护栏下优化策略模型。

为什么要KL：防止模型为了高奖励走极端（reward hacking），保持不偏离SFT太远。

工程组件：policy/ reference/ reward model（常配critic/value）；关键参数：采样温度、KL系数、reward归一化、优势估计。

典型坑：奖励被钻空子（套话高分）、训练不稳（语言质量下降）、过度保守（动不动拒答）；解法：对抗数据、多目标奖励、target-KL自适应、增加“该帮助”样本与拒答误杀率评估。

2.2 DPO：不用训练RM的偏好对齐捷径

直觉：用偏好对(x, y+, y-)直接训练，让模型提高y+相对y-的概率，并相对reference做约束。

工程更轻：只需要policy+ref，不需要RM与PPO rollout；关键在数据质量和hard negative（迷惑性负例）。

局限：更像监督学习，不擅长在线探索；多步工具任务往往还要Agentic RL。

2.3 GRPO：组内相对优势，贴近“只看结果就能学”

直觉：同一个prompt采样K个候选，reward打分后用组内相对优势更新，让高分候选概率变大。

优势：减少价值函数依赖，适合可自动打分任务（代码单测、数学判题、结构化输出）。

典型坑：退化组（候选都一样差→学习信号近0）；解法：选对domain、提高K、改进reward可靠性与探索策略。

3. Agentic RL：让模型“为了目标去做事”

区别：从单轮回答变成多步决策+工具调用+环境反馈。

工程闭环：定义可验证成功条件 → 定义工具接口 → 收集轨迹 → 先SFT学会用工具 → 再RL用成功率/成本优化。

常见问题：工具滥用、长链路信用分配难、环境不稳定、投机取巧骗奖励；解法：加入成本惩罚、分段奖励、可复现环境回放、用可验证reward（单测/规则）交叉校验。

4. 选择方法：一句话决策表

有高质量示范轨迹：先SFT。

有大量偏好对且想省工程：DPO。

能批量采样并可靠打分：GRPO/组采样类。

需要复杂约束与在线探索：RLHF(PPO)。

多步工具与目标完成：Agentic RL（环境反馈 + 可验证奖励）。

5. 自测思考题

偏好对齐的上限在哪里？如何避免迎合少数标注者偏见？

为什么Agentic RL更依赖可验证奖励？如何设计“可测试”的reward？

在固定算力下，你如何在“更好示范数据(SFT)”与“更多在线探索(RL)”之间分配预算？

Part B MiniMind (GitHub) 项目速览与上手

参考仓库：jingyaogong/minimind (GitHub)

MiniMind 项目速览

MiniMind 是一个面向学习与复现的开源项目，强调用尽量少的依赖从0实现训练全流程，包括 Pretrain、SFT、LoRA、DPO，以及基于AI反馈的强化学习（PPO/GRPO/SPO）等。

项目README中明确列出了：训练脚本在 trainer 目录、支持断点续训（--from_resume 1），以及用 eval_llm.py 通过 --weight 选择不同阶段权重进行评测。

它为什么适合作为“完备SFT + RLHF/RLAIF”学习仓库

可读性：尽量不用高度封装的第三方接口，便于你把“算法原理”和“代码实现”一一对上。

覆盖全流程：从数据集准备、SFT、偏好对齐（DPO）到在线强化学习（PPO/GRPO）。

工程细节：包含检查点恢复、单机/多卡训练方式示例，以及对RLAIF奖励模型放置路径的明确说明。

快速上手：典型训练命令（按README）

预训练：python train_pretrain.py

SFT：python train_full_sft.py （也可 torchrun --nproc_per_node 1 train_full_sft.py）

DPO：python train_dpo.py

PPO：python train_ppo.py （多卡示例：torchrun --nproc_per_node N train_ppo.py）

GRPO：python train_grpo.py （多卡示例：torchrun --nproc_per_node N train_grpo.py）

断点续训：为训练脚本添加 --from_resume 1 自动恢复 checkpoints

数据与目录要点

README说明：将下载的数据集文件放到 ./dataset/ 目录下；并提供了 sift_*.jsonl、dpo.jsonl、rlaif-mini.jsonl 等示例。

RLAIF 奖励模型：文档给出示例，建议将奖励模型（如 InternLM2-1.8B-Reward）放在与 minimind 仓库同级目录，形成 project/minimind 与 project/internlm2-1.8b-reward 的结构。

项目里对 DPO/PPO/GRPO 的“踩坑提示”

DPO：强调无需同步训练

Reward/Value，显存更低、收敛更稳，适合偏好/安全对齐，但在线探索有限。

PPO：文档提到 Actor-Critic 互依赖导致训练初期收敛可能偏慢，且显存占用更高。

GRPO：文档点出“退化组”问题：同组候选奖励几乎一样（尤其都很差）会导致学习信号接近0，因此需要选择在能力边界内的domain。

建议你用它学习（高效路径）

第1天：只跑 `eval_llm.py` 看不同权重差异（`pretrain/full_sft/dpo/ppo_actor/grpo`），建立直觉。

第2天：读 `train_full_sft.py` + 数据格式（`jsonl conversations`），搞懂模板与loss mask。

第3天：读 `train_dpo.py`，把DPO损失和你理解的“chosen vs rejected”对齐起来。

第4天：读 `train_ppo.py` /

`train_grpo.py`：只看“采样-打分-优势-更新-KL”这一条主线，别被工程细节淹没。

第5天：自己改一个reward：用规则/单测替换RM，做一个最小RLAIF实验。

常用命令速查

说明：下方命令块只保留可执行内容；解释放在正文里，避免字体兼容问题。

```
python train_pretrain.py
python train_full_sft.py
python train_dpo.py
python train_ppo.py
python train_grpo.py
python eval_llm.py --weight full_sft

# resume
python train_full_sft.py --from_resume 1
```

奖励模型目录示例（RLAIF）

说明：README建议把奖励模型放在与 `minimind` 仓库同级目录，便于脚本按相对路径加载。

```
project/
├── minimind/
│   ├── model/
│   └── ...
└── internlm2-1_8b-reward/
    ├── config.json
    ├── model.safetensors
    └── ...
```

参考与延伸阅读

1) MiniMind: <https://github.com/jingyaogong/minimind>

2) ReaLHF: <https://github.com/openpsi-project/ReaLHF>

说明：PDF内链接用于阅读与复现；口播时可不读链接。